2022 IEEE 17th International Conference on Computer Sciences and Information Technologies (CSIT) | 979-8-3503-3431-9/22/$31.00 ©2022 IEEE | DOI: 10.1109/CSIT56902.2022.10000626

A statistics-based performance testing methodology: a case study for the I/O bound tasks

Andriy Sultanov*, Maksym Protsyk*, Maksym Kuzyshyn*, Daria Omelkina*, Vyacheslav Shevchuk†, Oleg Farenyuk*‡
* *Faculty of Applied Sciences of Ukrainian Catholic University, L'viv, Ukraine*
† *Dialog Semiconductor, L'viv, Ukraine*
‡ *Institute for Condensed Matter Physics of the National Academy of Sciences of Ukraine, L'viv, Ukraine*
e-mail: indrekis@icmp.lviv.ua

***Abstract*—This research demonstrates the applicability of the established statistical methodology developed for experimental data analysis within physics, biology, and other natural science disciplines to the analysis of computing performance measurements. The methodology is based on the randomization of test samples and test conditions as well as usage of statistical hypothesis testing to determine if the performance differs. The hypothesis testing method is selected based on the statistical distribution of the data—Student's t-test for the normal distribution and non-parametric tests, such as the Wilcoxon signed-rank test, for other distributions. Multiple comparisons problem is taken into account using the Bonferroni correction. Though other important natural science methods exist—different kinds of blinded experiments and analysis could be beneficial too—they could be too expensive for such contexts without additional justifications. The method will be applied to a model problem: a comparison of the performance of three implementations of web crawlers, compiled with different options and different compilers. Experimental cases were intentionally selected with both large and minor (but definite) performance differences to show how statistical methods can help determine the appearance of the performance difference and confidently discuss its extent to take into account benefits or drawbacks of the various setups.**

***Index Terms*—Web crawlers, parallel and concurrent programming, benchmarking, multithreading, epoll, Wilcoxon signed-Rank Test, Student t-test, Bonferroni correction**

I. Introduction

Performance measurements are essential for selecting optimal hardware, algorithms, software instruments (i.e., compilers, libraries, frameworks), and so on. Though, correct and reproducible performance analysis is a complex task. There are many reasons for this: the complex nature of the modern hardware and software stacks, limited introspection abilities [1], [2], inherited randomness of many processes in modern computer systems, and so on.

This work is dedicated to methods that take into account the randomness of the measurements. In order to draw conclusions from comparisons of experimental data, these data should be prepared and processed in a unified and consistent way. For such analysis, we propose a framework providing correct and justified results. Such methods are widely used in the natural science experimental data analysis but rarely used in computer systems performance studies though the topic is discussed in the literature [3], [4].

A good sample for our approach in a computer science domain is benchmarking the execution time of input/output-bound programs tested under predefined conditions. We selected web crawlers as examples of the I/O-bound programs.

For the test, we compare three crawlers, each time applying and combining various parameters, such as different C++ compilers, optimization level, turned on and turned off hyper-threading (HT) and Intel Turbo Boost Technology (TB). As a comparison metric, we use the time of execution of these programs – wall time.

This paper will primarily focus on the details of the testing framework for this exact problem, but such a pipeline can be applied to other types of experimental data, too.

II. Crawlers' architecture and implementation

Web crawlers are an essential part of the various search engines. They, starting from some predefined set of URLs, load the pages, parse them and often recursively load linked pages. To simplify the performance comparison of various methods making it more predictable, we decided to omit recursive behavior, opting for crawling a set of predefined web pages.

Crawlers used in this work are based on a single abstract crawler, which implements the elements common to different methods. It provides elementary operations: add a URL to the queue, read a list of predefined URLs from a file, get a resulting HTML file, simulate parsing the HTML by collecting its tags. It also allows different crawlers' implementations to provide their definitions of the method that processes the incoming URL queue.

Several basic parallelization methods were used:

1) The workload is assigned to the process pool with as many processes as needed.
2) The workload is assigned to the thread pool with as many threads as needed.
3) M:N model, where each of the M threads is assigned many tasks and processes them asynchronously. A corresponding state machine was implemented using the epoll system call.

In order to focus on the difference in performance, we decided to minimize the communication between processes or threads so each of them processes its own set of URLs independently. Though such an approach is to some extent

artificial, it makes the experimental system much more predictable.

We implemented crawlers using the C++ language. All the crawlers use the same requests infrastructure, based on POSIX (Berkeley) socket API and corresponding helper libraries (netdb, arpa/inet). Implementations of the main abstract crawler, three types of crawlers for testing, a description of the used server setup for the Ubuntu Linux distribution, and other additional helper modules are available in the GitHub repository of the research [5]. A more detailed description of the crawlers' implementation is available in our previous work [6].

III. The analysis pipeline

Experimental data analysis in physics and other natural sciences has an established data processing pipeline. In this work, we propose its adaptation to the performance measurements. Though exact steps depend on performance research goals, the general data processing pipeline has the following steps:

1) Input data preparation:
   a) **Randomization**: The input data for the analyzed program should be randomized before proceeding.
2) Performing experiments and obtaining results.
3) Preprocessing results:
   a) Removing outliers using interquartile range (IQR).
   b) Testing normality of the data distribution.
4) **Bonferroni correction**. After selecting the desired p-value as the criterion of a significant difference — α, divide it by the number of the comparisons m and use this new value $\alpha_c = \alpha/m$ for the tests.
5) Further steps depend on the results of this test.
6) If the distribution is normal (Gaussian):
   a) Performing Student's t-test pairwise.
   b) Calculating the pairwise relative difference between sample **means** to determine performance difference.
7) If the distribution is not normal:
   a) Performing Wilcoxon non-parametric test.
   b) Calculating the pairwise relative difference between sample **minimums** to determine performance difference.
8) As a double check, even for the (presumably) normal distribution, one can additionally perform Wilcoxon non-parametric test.

A. Justification of the chosen approaches: important details

1) Randomization: It is a widely used practice in natural science and computer science is becoming aware of the problem [7]. Randomizing reduces bias, equalizing factors not accounted for in the experiment. Without it, results could contain a large bias rendering them useless. A random shuffling of the HTML links is used for our model task. If the program does not have input, but the execution is the experiment itself, the execution order should be randomized too [7].

2) Removing outliers: Although outliers can be produced by systematic errors or experiment flaws, large enough datasets always contain outliers, and discarding them benefits further statistical analysis [8]. One should be careful not to discard correct data described by multi-modal or long-tail distributions as outliers.

3) Bonferroni correction: Performing extensive comparison, for example, while investigating the performance of some approaches, one should be aware of the multiple comparisons problem also called "look-elsewhere effect" [9] — if there are enough pairwise comparisons, there will be some accidental differences between the compared experiments for any selected significance (p-value). In other words, the more comparisons are made, the more statistical significance of their difference is overestimated. For performance experiments, it seems that the Bonferroni correction is the best way to deal with multiple comparisons — it is simple, rather conservative, and does not depend on the analyzed data distribution [10].

4) Normality testing: The normal test checks whether the sample comes from the normal distribution or not. We choose the Ralph D'Agostino's and E. S. Pearson's test [11]. It uses kurtosis and skewness of the sample.

5) Student t-test: A two-sided test for two independent samples was selected. The null hypothesis is that these samples have the same average values. The t-test determines the difference between the arithmetic means of the given samples and therefore allows one to understand if these samples come from the same distribution [12].

6) Wilcoxon rank-sum test: Just as the Student t-test, it is used to test a null hypothesis that the samples are from the same distribution. The important difference — the Wilcoxon rank-sum test is non-parametric and is suitable for testing values from unknown distributions [13], [14].

7) Technical note: For this investigation, we used implementations of the statistical methods from the SciPy.

IV. Experimental setup

Our work uses existing algorithms for crawling to show how our testing approach works on similar programs under different circumstances. To emphasize the benefits of the proposed framework, we intentionally choose experiments with small (if present at all) expected differences, and expected distributions could also be substantially non-normal.

The crawlers were tested for the different combinations of the compilers, optimization levels, hardware technologies enabled, and CPU used. Details are presented in the Table I. We used a predefined set of 130000 HTML files for testing, which can be downloaded at [15].

NGINX was used as an HTML server on a separate machine connected to the client computer by the direct Gigabit Ethernet connection. Files are stored on the dedicated RAM disk. TLS was not used, and the data compression was disabled on the server's side. CPU, memory, and bandwidth utilization of the server machine were controlled to avoid starvation on the server side.

TABLE I
EXPERIMENT OPTIONS USED AND TEST SETUP

Compiler	GCC	Clang			
Version	11	12			
Optimisation	-O0	-O1	-O2	-O3	-O0 -march=native
Turbo Boost	On	Off			
Hyper-threading	On	Off			

	Client	***Server***
CPU	Intel i5-8265U	Intel i5-2500
CPU cores	4	4
Base frequency	1.60 GHz	3.3 GHz
Hyperthreading	disabled	absent
RAM	16 Gb DDR4	24 Gb DDR4
RAM frequency	2667 MHz	2133 MHz
Operating system	Linux Ubuntu 20.04 LTS	

Execution time was measured using the google-benchmark framework. The Linux perf profiler was used as a control for the wall-time measurements. Outliers were removed using 1.5 IQR.

V. EXPERIMENTAL RESULTS AND THEIR ANALYSIS

Experiments were performed for all combinations of the parameters (Table I). To demonstrate the benefits of the proposed framework we chose to compare:

- Performance of different types of crawler implementations for the GCC and `-O3` optimization. Turbo Boost and hyper-threading are disabled. This is the most obvious comparison — results can be easily predicted [16].
- Effect of the Turbo Boost and hyper-threading on each of the crawlers implementation.
- Performance difference of the optimization levels for the GCC and Clang. Turbo Boost and hyper-threading are disabled.
- Performance of the crawler implementations with Clang `-O0`, with and without `-march=native` option. Turbo Boost and hyper-threading are disabled.

A. Performance of different types of crawler implementations

In the considered setup, distributions are normal with p-value $= 0.01$ (though epoll results are close to being non-normal, $p = 0.03$). so we can use the Student's t-test pairwise for all crawlers to check the null hypothesis that the data from different crawlers comes from the same distributions, and there is no statistically significant difference in their performance.

TABLE II
THE RELATIVE PERFORMANCE OF CRAWLER IMPLEMENTATIONS. GCC WITH -O3. TURBO BOOST AND HYPER-THREADING ARE DISABLED. COLUMN REPRESENT HOW THIS CRAWLER IS PERFORMING RELATIVE TO THE ONE IN THE ROW: $((t_{row} - t_{column})/\min(t_{row}, t_{column})) \cdot 100$, WHERE t_{row} AND t_{column}, ARE WORKING TIMES FOR THE CORRESPONDING ROW OR COLUMN CRAWLER.

	process pool	thread pool	epoll
process pool	0	$+2.2\%$	$+4.4\%$
thread pool		0	$+2.3\%$
epoll			0

We reject the null hypotheses that each pair of crawler's data are not significantly different from each other—their performance substantially differs. The pairwise relative difference between the sample mean times is presented in Table II.

Results are expected—though the epoll-based approach is more challenging to implement, it gives the best performance, and the process per socket approach is slowest though it is the most secure and easiest to implement.

B. Effect of the Turbo Boost and the hyper-threading

One can expect that Turbo boost or hyper-threading technologies would have different impacts on the crawlers of different architecture. Turning on both could also scale non-linearly — hyper-threading uses more CPU blocks, leading to higher energy consumption per physical core and limited maximal frequency, among others, by the ability to dissipate excessive heat. To check this impact, crawlers were compiled by GCC using `-O3` optimization supposedly using more of the CPU's transistors.

Results are presented in the Table III. All distributions are normal except for the thread pool crawler when the Turbo Boost is turned off (p-value less than 10^{-4}).

TABLE III
RELATIVE PERFORMANCE OF EACH FOR TURBO BOOST (TB) AND HYPER-THREADING (HT) ENABLED OR DISABLED. GCC WITH -O3. ALL DIFFERENCES ARE STATISTICALLY SIGNIFICANT (P-VALUES $< 10^{-7}$). THE NOTATION USED IS THE SAME AS FOR TABLE II.

	No TB, no HT	TB, no HT	no TB, HT	TB, HT
epoll				
No TB, no HT	0	$+16.95\%$	$+13.68\%$	$+29.63\%$
TB, no HT		0	-3.79%	$+15.27\%$
no TB, HT			0	$+18.48\%$
TB, HT				0
Thread pool				
No TB, no HT	0	$+16.28\%$	$+15.35\%$	$+29.23\%$
TB, no HT		0	-1.2%	$+15.43\%$
no TB, HT			0	$+16.44\%$
TB, HT				0
Process pool				
No TB, no HT	0	$+16.49\%$	$+15.38\%$	$+29.55\%$
TB, no HT		0	-1.32%	$+15.64\%$
no TB, HT			0	$+16.75\%$
TB, HT				0

One can see that both Turbo Boost and hyper-threading are beneficial to a similar extent for each of the crawlers, and their impacts are combined almost linearly. One can note that even for such embarrassing parallel tasks, while logical cores numbers grow two times when hyper-threading is turned on, performance grows only in order of the 15%.

C. Performance difference of optimization levels

Performance differences for the code, generated by the different compilers or by the same compiler for different optimization options, is the hot question and provokes many discussions. Results of the comparison here are in no way conclusive to compare compilers under the investigation — this would require a comparison of much more diverse code.

TABLE IV
THE RELATIVE PERFORMANCE OF THE PROCESS POOL CRAWLER FOR THE DIFFERENT OPTIMIZATION OPTIONS OF THE GCC AND CLANG. NO TB OR HT. NUMBERS IN THE COLUMN REPRESENT HOW THIS CONFIGURATION PERFORMS RELATIVE TO THE CONFIGURATION IN THE ROW. THE NOTATION USED IS THE SAME AS FOR TABLE II. "*No diff.*" – THE DIFFERENCE IS STATISTICALLY INSIGNIFICANT.

		GCC				Clang			
		-O0	-O1	-O2	-O3	-O0	-O1	-O2	-O3
GCC	O0	0	-0.17%	-0.11%	-0.18%	No diff.			
	O1		0	No diff.	No diff.		$+0.11\%$		
	O2			0	No diff.			No diff.	
	O3				0				$+0.17\%$
Clang	O0					0	$+0.30\%$	$+0.19\%$	$+0.34\%$
	O1						0	-0.10%	No diff.
	O2							0	$+0.15\%$
	O3								0

Moreover, heavily input-output bound code often benefits less from the compiler optimizations. Here our sole intention is to show how the proposed approach could be useful to boost compiler optimization performance.

Results for the process pool crawler are presented in Table IV. The exact results are somewhat unexpected — for example, GCC optimizations even slightly slow down the code. This result was thoroughly checked and reproduced, but the exact causes of this behavior are yet to be determined. It is important that the proposed framework can detect performance differences as small as 0.1% — the p-value for this value here equals $2 \cdot 10^{-5}$ and the difference remains significant even for rather conservative Bonferroni correction factor $m = 100$.

D. Effect of the -march=native option

Compilers mostly use some common subset of instructions of the x86 architecture. For compilers used in this research, it is the common subset of the x86-64 instructions, including SSE2, but not SSE3 and newer instruction set extensions. Option `-march=native` instructs the compiler to use all available instructions for the CPU used to compile the code. Results are presented in Table V. One can see that some differences (though small, expectantly for such types of tasks) are still present.

TABLE V
THE RELATIVE PERFORMANCE OF CRAWLERS FOR -MARCH=NATIVE AND DEFAULT INSTRUCTION SET USED BY THE CLANG WITH -O0. NO TB OR HT.

Crawler	Performance effect of -march=native	p-value
epoll	Statistically insignificant	0.26
thread pool	$+0.1\%$	$3 \cdot 10^{-7}$
process pool	$+0.29\%$	$2 \cdot 10^{-14}$

VI. CONCLUSION

The proposed framework, inspired by the approach of experimental data analysis used in natural science, allows reliable comparison of the performance impact of factors studied while working with noisy data. Input should always be randomized, and outliers should be removed before further analysis. Execution time measurements are sometimes described by the non-normal (non-Gaussian) distributions. One should consider this before using statistical hypothesis testing. We demonstrate, using several cases, that when one can perform enough tests for each setup, this approach allows one to reliably notice even a small performance impact of some controlled factor, even if this impact is smaller than the noise-caused variations of the execution time.

REFERENCES

[1] V. M. Weaver and J. Dongarra, "Can hardware performance counters produce expected, deterministic results?" in *Proceedings of Third Workshop on Functionality of Hardware Performance Monitoring*, Atlanta, GA, 12 2010.

[2] B. Gregg, "Linux systems performance," in *Proceedings of LISA19*. Portland, OR: USENIX Association, Oct. 2019.

[3] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous java performance evaluation," *ACM SIGPLAN Notices*, vol. 42, no. 10, p. 57, oct 2007.

[4] L. Bulej, V. Horký, and P. Tůma, "Do we teach useful statistics for performance evaluation?" in *Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering Companion*, ser. ICPE '17 Companion. New York, NY, USA: Association for Computing Machinery, 2017, p. 185–189.

[5] "Code and data for this research," 2022. [Online]. Available: https://github.com/dariaomelkina/parallel-crawling-research

[6] A. Sultanov, M. Protsyk, M. Kuzyshyn, D. Omelkina, V. Shevchuk, and O. Farenyuk, "Comparison of performance of the popular approaches to implementing parallel crawlers," in *2021 IEEE 16th International Conference on Computer Sciences and Information Technologies (CSIT)*, vol. 1, 2021, pp. 349–352.

[7] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" *ACM Sigplan Notices*, vol. 44, no. 3, pp. 265–276, 2009.

[8] A. Zimek and P. Filzmoser, "There and back again: Outlier detection between statistical reasoning and data mining algorithms," *WIREs Data Mining and Knowledge Discovery*, vol. 8, no. 6, p. e1280, 2018.

[9] J. P. Shaffer, "Multiple hypothesis testing," *Annual Review of Psychology*, vol. 46, no. 1, pp. 561–584, 1995.

[10] J. Dunn and O. J. Dunn, "Multiple comparisons among means," *Journal of the American statistical association*, vol. 56, no. 293, pp. 52–64, 1961.

[11] R. D'Agostino and E. S. Pearson, "Tests for departure from normality. empirical results for the distributions of b_2 and $\sqrt{b_1}$," *Biometrika*, vol. 60, no. 3, pp. 613–622, 1973.

[12] W. S. Gosset, "The probable error of a mean," *Biometrika*, vol. 6, no. 1, pp. 1–25, March 1908, under the pseudonym "Student".

[13] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[14] D. Rey and M. Neuhäuser, *Wilcoxon-Signed-Rank Test*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 1658–1659.

[15] "Dataset with HTML-files collected for the research," 2021. [Online]. Available: https://kutt.it/STXM97

[16] P. Karabyn, "Performance and scalability analysis of java io and nio based server models, their implementation and comparison," 2019, ukrainian Catholic University bachelor diploma thesis.